Universidad de los Andes

Departamento de Ingeniería Industrial · Modelado de Sistemas bajo Incertidumbre

Verificación y Validación de Modelos de Simulación

Alejandra Tabares Pozos

Índice

1. La distinción fundamental: verificar vs. validar **3**

2. Marco de Sargent para V&V **3**

3. Proceso integrado de V&V **4**

4. Técnicas de verificación **4**

4.1. Revisión estructurada del código 4

4.2. Traza de una entidad (entity trace) 4

4.3. Pruebas con entradas degeneradas 5

4.4. Conservación de entidades 5

4.5. Pruebas de simplificación 5

4.6. Uso de aserciones en el código 5

4.7. Verificación con semilla determinista 6

5. Técnicas de validación **6**

5.1. Validación de cara (face validity) 6

5.2. Test de Turing 6

5.3. Validación con datos históricos 6

5.4. Validación estadística con soluciones analíticas 7

5.5. Comparación estadística: prueba t e intervalos de confianza 7

6. Análisis de sensibilidad como herramienta de validación **8**

7. Credibilidad y documentación del modelo **8**

8. Ejemplo básico: verificación de una cola $M/M/1$ **9**

8.1. Prueba degenerada 9

8.2. Conservación de entidades 9
8.3. Traza de eventos 9
8.4. Prueba de tráfico extremo 10

9. Ejemplo intermedio: validación del $M/M/1$ y detección de un error 10
9.1. Validación del modelo correcto ($\mu = 5$) 10
9.2. Detección de un bug: $\mu = 4$ en vez de $\mu = 5$ 11

10. Ejemplo escalado: celda de manufactura $M/E_2/1$ con análisis de sensibilidad 11
10.1. Verificación estructural 12
10.2. Validación operacional con la fórmula P-K 12
10.3. Análisis de sensibilidad 12

1. La distinción fundamental: verificar vs. validar

Construir un modelo de simulación involucra tres capas: el **sistema real**, el **modelo conceptual** (descripción matemática del sistema) y el **modelo computacional** (implementación en código). Dos preguntas distintas emergen:

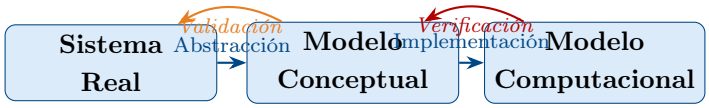
Verificación

¿Construimos el modelo correctamente? La verificación comprueba que el modelo computacional implementa fielmente el modelo conceptual. Es un problema de *ingeniería de software*: ausencia de bugs, lógica correcta, parámetros bien cargados.

Validación

¿Construimos el modelo correcto? La validación comprueba que el modelo conceptual (y por ende el computacional) representa con suficiente fidelidad el sistema real para los propósitos del estudio. Es un problema de *modelado*: ¿captura el comportamiento relevante?

Un modelo puede estar verificado (sin bugs) pero no validado (con supuestos incorrectos), y viceversa. Ambos procesos son necesarios.



2. Marco de Sargent para V&V

Sargent (2013) propone el marco de referencia más citado en la disciplina, que organiza la V&V en tres tipos de validez:

1. **Validez conceptual (conceptual validity)**: ¿es razonable la teoría y la estructura del modelo para los propósitos del estudio? Se evalúa revisando los supuestos, las distribuciones de entrada (Tema 3.1) y la lógica de eventos.

2. **Validez computacional (computerized model verification)**: ¿la implementación reproduce fielmente el modelo conceptual? Esto es la verificación.

3. **Validez operacional (operational validity)**: ¿las salidas del modelo computacional coinciden con las del sistema real dentro de un margen aceptable? Esto es la validación propiamente dicha.

Adicionalmente, la **validez de datos** cruza las tres capas: los datos usados para construir, calibrar y validar el modelo deben ser suficientes y correctos (Tema 3.1).

V&V es un proceso continuo

La V&V no es un paso único al final del proyecto. Cada modificación al modelo (nueva distribución, nuevo tipo de evento, cambio de parámetro) requiere re-verificación y potencialmente re-validación. Un modelo validado para un rango de operación puede no ser válido fuera de ese rango.

3. Proceso integrado de V&V

Proceso integrado de V&V — 6 pasos

1. **Validación de cara (face validity)**: ¿es razonable el modelo desde el punto de vista del experto de dominio?
2. **Verificación estructural**: revisión del código, trazas de entidades, pruebas extremas.
3. **Verificación de eventos**: comprobar que cada tipo de evento actualiza el estado correctamente.
4. **Validación operacional**: comparar salidas del modelo con datos históricos o soluciones analíticas conocidas.
5. **Validación estadística**: pruebas de hipótesis que comparan métricas del modelo con métricas del sistema real.
6. **Análisis de sensibilidad**: verificar que el modelo responde en la dirección y magnitud esperadas ante cambios en los parámetros de entrada.

4. Técnicas de verificación

4.1. Revisión estructurada del código

Un segundo analista revisa el código independientemente del autor. Se verifican:

- Que cada evento actualice *todos* los contadores de estado relevantes.
- Que la lógica de la cola (FIFO, LIFO, prioridades) coincida con el modelo conceptual.
- Que los contadores de estadísticas (área bajo $N(t)$, tiempos individuales) se actualicen en el momento correcto.
- Que los parámetros de entrada (λ, μ, c, K) estén correctamente cargados y no intercambiados.

4.2. Traza de una entidad (entity trace)

Se ejecuta la simulación con un flujo controlado y se imprime el estado del sistema en cada evento. La traza debe coincidir exactamente con el cálculo manual.

Formato de tabla de traza

Para cada evento, registrar:

1. Tiempo del evento (t).
2. Tipo de evento (llegada, inicio de servicio, salida).
3. Entidad involucrada (ID del cliente).
4. Estado del sistema después del evento: $N(t)$ (número en sistema), estado del servidor, contenido de la cola.
5. Contadores actualizados: tiempo de espera acumulado, área bajo $N(t)$.

4.3. Pruebas con entradas degeneradas

- **Sin aleatoriedad**: usar tiempos de llegada y servicio deterministas conocidos. La salida debe ser exactamente calculable a mano.
- **Tráfico cero** ($\lambda \to 0$): el sistema siempre estará vacío, $W_q \to 0$, $\rho \to 0$.
- **Tráfico extremo** ($\rho \to 1$): la cola debe crecer sin límite; W_q y L_q deben diverger.
- **Capacidad infinita vs. finita**: al fijar K (capacidad), el flujo de salida debe ser $\leq \lambda$ por el rechazo de clientes.

4.4. Conservación de entidades

En toda simulación de cola, la identidad:

$$\text{Llegadas totales} = \text{Salidas totales} + \text{Entidades en sistema al final}$$

debe cumplirse exactamente. Si no se cumple, hay un bug en la lógica de eventos.

4.5. Pruebas de simplificación

Si el sistema tiene c servidores idénticos con la misma tasa, una simulación de c servidores debería producir resultados estadísticamente equivalentes a c subsistemas independientes de un solo servidor (bajo ciertas condiciones de enrutamiento). Esta consistencia es fácilmente verificable.

4.6. Uso de aserciones en el código

Las **aserciones** (assertions) son condiciones que deben ser verdaderas en todo momento durante la ejecución. Son la verificación automatizada más eficiente:

- $N(t) \geq 0$ siempre (el número en sistema no puede ser negativo).
- Si el servidor está libre y la cola no está vacía, se inicia servicio inmediatamente.
- El tiempo del próximo evento es siempre $\geq$ el tiempo actual (no hay viajes al pasado).

- Los tiempos de servicio generados son > 0.
- Al finalizar un servicio, el cliente atendido es el correcto según la disciplina de cola.

Una aserción violada detiene la simulación y señala exactamente dónde está el error, lo cual es mucho más eficiente que buscar bugs observando salidas incorrectas.

4.7. Verificación con semilla determinista

Fijar la semilla del generador de números aleatorios permite reproducir exactamente la misma corrida. Esto es esencial para:

- Depurar: si una corrida produce un resultado sospechoso, se puede reproducir y trazar paso a paso.
- Comparar: dos implementaciones del mismo modelo con la misma semilla deben producir resultados idénticos.
- Regresión: después de modificar el código, verificar que las corridas anteriores se reproducen exactamente (si no se cambió la lógica que las afecta).

5. Técnicas de validación

5.1. Validación de cara (face validity)

Presentar el modelo a expertos del dominio (operadores, gerentes) y preguntarles si el comportamiento del modelo es razonable. Esta es la primera línea de defensa. Preguntas clave:

- ¿Los tiempos de ciclo simulados son similares a los observados informalmente?
- ¿Las situaciones de congestión ocurren en los mismos momentos del día?
- ¿Las métricas de utilización de servidores parecen realistas?

5.2. Test de Turing

Un experto del dominio recibe un conjunto de datos reales y un conjunto de datos simulados (mezclados o separados) y debe identificar cuál es cuál. Si no puede distinguirlos, el modelo tiene alta fidelidad operacional.

5.3. Validación con datos históricos

Se divide el registro histórico en dos partes: un periodo para **calibrar** el modelo (ajustar distribuciones y parámetros) y otro para **validar** (comparar métricas del modelo con las del sistema en ese periodo).

$$\text{Error relativo} = \frac{|\hat{\theta}_{\text{sim}} - \theta_{\text{real}}|}{|\theta_{\text{real}}|} \times 100\,\%$$

Un error relativo por debajo de un umbral predefinido (típicamente 5–10 %) indica validez operacional suficiente para el propósito del estudio.

5.4. Validación estadística con soluciones analíticas

Cuando el sistema es suficientemente simple, soluciones analíticas exactas sirven como **punto de referencia de oro**. Las principales fórmulas de validación son:

Modelo	**Métrica**	**Fórmula exacta**
$M/M/1$	W_q (tiempo en cola)	$\dfrac{\rho}{\mu(1-\rho)}$
$M/M/1$	L_q (clientes en cola)	$\dfrac{\rho^2}{1-\rho}$
$M/M/1$	ρ (utilización)	λ/μ
$M/G/1$	W_q (P-K)	$\dfrac{\lambda\mathbb{E}[S^2]}{2(1-\rho)}$
$M/M/c$	ρ (utilización)	$\lambda/(c\mu)$

La fórmula de Pollaczek–Khinchine como herramienta de validación

La cola $M/G/1$ tiene solución analítica para W_q (y L_q por la ley de Little) pero no para la distribución completa del tiempo de espera. Esto la convierte en una herramienta ideal de validación: podemos verificar el promedio con la fórmula P-K y, si coincide, confiar en que la simulación es correcta para las métricas más complejas (percentiles, probabilidad de espera $> t$).

5.5. Comparación estadística: prueba t e intervalos de confianza

Si se tienen n_R réplicas del simulador, obteniendo promedios $Y_1, \ldots, Y_{n_R}$, y un valor de referencia θ_0 (analítico o del sistema real):

Prueba t:

$$H_0 : \mu_{\text{sim}} = \theta_0 \quad \text{vs.} \quad H_1 : \mu_{\text{sim}} \neq \theta_0$$

$$t = \frac{\bar{Y}_{n_R} - \theta_0}{S_{n_R}/\sqrt{n_R}} \sim t_{n_R-1} \quad \text{bajo } H_0$$

Rechazar H_0 si $|t| > t_{\alpha/2, n_R-1}$.

Enfoque por intervalo de confianza (equivalente y más informativo):

$$IC_{1-\alpha}(\mu_{\text{sim}}) = \bar{Y}_{n_R} \pm t_{\alpha/2,n_R-1} \cdot \frac{S_{n_R}}{\sqrt{n_R}}$$

Si el IC contiene θ_0, el modelo es consistente con la referencia. La ventaja del IC sobre la prueba t es que muestra la *magnitud* de la discrepancia, no solo su significancia.

Significancia estadística vs. significancia práctica

Con muchas réplicas, una diferencia de 0,01 min entre el modelo y la referencia puede ser estadísticamente significativa (IC estrecho que no contiene θ_0) pero prácticamente irrelevante. Siempre evaluar si la discrepancia tiene consecuencias operativas. Un modelo con error relativo $< 5\,\%$ es generalmente aceptable para toma de decisiones.

6. Análisis de sensibilidad como herramienta de validación

El análisis de sensibilidad verifica que el modelo responde *en la dirección correcta* y *con magnitud razonable* ante cambios en los parámetros de entrada. Es una prueba de "sentido común cuantitativo".

Procedimiento de análisis de sensibilidad

1. Seleccionar los parámetros clave del modelo (p.ej. λ, μ, c).
2. Definir un rango de variación: típicamente $\pm 10\,\%$ o $\pm 20\,\%$ alrededor del valor base.
3. Para cada variación, ejecutar n_R réplicas y registrar la métrica de salida.
4. Verificar:
   - **Dirección**: si λ aumenta, W_q debe aumentar (para $\rho < 1$).
   - **Magnitud**: la respuesta debe ser coherente con la teoría de colas (no lineal cerca de $\rho \to 1$).
   - **Monotonía**: si c aumenta (más servidores), W_q debe disminuir.
5. Si alguna dirección es incorrecta, hay un **error en el modelo**.

El análisis de sensibilidad no solo valida; también identifica qué parámetros tienen mayor impacto en la salida, lo que orienta dónde invertir más esfuerzo en el modelado de entradas (Tema 3.1).

7. Credibilidad y documentación del modelo

Un modelo de simulación solo es útil si quienes toman decisiones *confían* en él. La credibilidad se construye con:

- **Documentación del modelo conceptual**: supuestos, distribuciones elegidas (con justificación del Tema 3.1), simplificaciones y su justificación.

- **Registro de V&V**: todas las pruebas realizadas, sus resultados, y las acciones correctivas tomadas. Un informe de V&V típico incluye:
  1. Descripción del modelo y sus supuestos.
  2. Pruebas de verificación realizadas (trazas, conservación, degeneradas).
  3. Comparaciones con soluciones analíticas (tablas de IC).
  4. Resultados de validación con datos históricos (si disponibles).
  5. Análisis de sensibilidad.
  6. Limitaciones conocidas del modelo.
- **Participación de stakeholders**: involucrar a los expertos de dominio en el proceso de V&V aumenta la confianza en los resultados.

8. Ejemplo básico: verificación de una cola $M/M/1$

Sistema

Se implementa una simulación de cola $M/M/1$ con $\lambda = 3$ clientes/min y $\mu = 5$ clientes/min ($\rho = 0{,}6$). Antes de usar el modelo para análisis de políticas, se aplica la verificación estructural.

Valores analíticos de referencia:

$$W_q = \frac{0{,}6}{5 \times 0{,}4} = 0{,}30 \text{ min}, \quad L_q = \frac{0{,}6^2}{0{,}4} = 0{,}90 \text{ clientes}, \quad \rho = 0{,}60$$

8.1. Prueba degenerada

Se fija $\lambda = 0$ (sin llegadas). La simulación reporta $N(t) = 0$, $W_q = 0$, $\rho = 0$ exactamente durante $T = 1000$ min. ✓

8.2. Conservación de entidades

En una corrida de $T = 1000$ min se cuentan 3 012 llegadas y 3 012 salidas (sistema vacío al final). La identidad $3012 = 3012 + 0$ se cumple. ✓

8.3. Traza de eventos

Se ejecuta con semilla fija y se registran los primeros 6 eventos:

Evento	t (min)	**Tipo**	**Cliente**	**Servidor**	$N(t)$	**Cola**	W_q cliente
1	0,28	Llegada	C1	Libre → Ocup.	1	$\emptyset$	—
2	0,47	Salida	C1	Ocup. → Libre	0	$\emptyset$	0,00
3	0,64	Llegada	C2	Libre → Ocup.	1	$\emptyset$	—
4	0,79	Salida	C2	Ocup. → Libre	0	$\emptyset$	0,00
5	0,91	Llegada	C3	Libre → Ocup.	1	$\emptyset$	—
6	0,98	Llegada	C4	Ocupado	2	{C4}	—

Verificación manual: C1 llega a $t = 0{,}28$; su tiempo de servicio generado es 0,19 min, así que sale a $t = 0{,}28 + 0{,}19 = 0{,}47$. ✓. C2 llega a $t = 0{,}64$ con servidor libre, servicio de 0,15 min, sale a $t = 0{,}79$. ✓. C4 llega a $t = 0{,}98$ con servidor ocupado (C3 aún en servicio), entra a la cola. ✓

8.4. Prueba de tráfico extremo

Con $\lambda = 4{,}99$ y $\mu = 5$ ($\rho = 0{,}998$), la cola crece sin límite: L_q supera 500 clientes en $T = 5000$ min. ✓

Resultado de la verificación

Todas las pruebas de verificación pasan: conservación de entidades, traza manual consistente, pruebas degeneradas correctas, y comportamiento extremo coherente. El código implementa fielmente el modelo conceptual $M/M/1$. Procedemos a la validación en el ejemplo intermedio.

9. Ejemplo intermedio: validación del $M/M/1$ y detección de un error

Sistema

Continuando con el mismo sistema $M/M/1$ ($\lambda = 3$, $\mu = 5$), ahora se ejecutan réplicas para **validar** contra las fórmulas analíticas. Además, se muestra qué ocurre cuando el código tiene un bug sutil: el programador accidentalmente usó $\mu = 4$ en lugar de $\mu = 5$ en la generación de tiempos de servicio.

9.1. Validación del modelo correcto ($\mu = 5$)

Se ejecutan $n_R = 30$ réplicas de $T = 2000$ min:

Métrica	**Analítico**	$\bar{Y}_{30}$	S_{30}	**IC 95 %**	**Error rel.**
W_q (min)	0,300	0,308	0,041	[0,293; 0,323]	2,7 %
L_q (cl.)	0,900	0,924	0,126	[0,877; 0,971]	2,7 %
ρ	0,600	0,598	0,006	[0,596; 0,600]	0,3 %

Todos los IC contienen el valor analítico. El modelo está **validado**.

9.2. Detección de un bug: $\mu = 4$ en vez de $\mu = 5$

Supongamos que el programador, por un error de transcripción, codificó la tasa de servicio como $\mu = 4$ en lugar de $\mu = 5$. Esto da $\rho_{\text{real}} = 3/4 = 0{,}75$ en vez de 0,60. El analítico incorrecto sería:

$$W_q^{\text{bug}} = \frac{0{,}75}{4 \times 0{,}25} = 0{,}75 \text{ min} \quad \text{(vs. 0,30 min esperado)}$$

Las 30 réplicas con el bug producen:

Métrica	**Ref.** ($\mu = 5$)	$\bar{Y}_{30}$	**IC 95 %**	**¿Contiene ref.?**	**Diagnóstico**
W_q (min)	0,300	0,762	[0,701; 0,823]	**No**	Bug detectado
ρ	0,600	0,749	[0,745; 0,753]	**No**	μ incorrecto

El IC de W_q no contiene 0,300, y el de ρ no contiene 0,600. La validación **detecta el error**. Nótese que $\hat{\rho} \approx 0{,}75 = 3/4$, lo que apunta directamente a que la tasa de servicio es 4 en lugar de 5.

La validación detecta errores que la verificación puede no encontrar

La verificación (trazas, conservación, pruebas degeneradas) habría pasado perfectamente con $\mu = 4$: el código funciona correctamente, solo que con el parámetro equivocado. Es la comparación con la referencia analítica la que revela la discrepancia. Esto muestra por qué **ambos procesos** (verificación y validación) son necesarios.

Resultado del ejemplo intermedio

La validación contra fórmulas analíticas es la herramienta más poderosa para detectar errores de parametrización. En el modelo correcto ($\mu = 5$), los IC al 95 % contienen todos los valores de referencia (error relativo $< 3\,\%$). Con el bug ($\mu = 4$), el IC de ρ apunta inmediatamente al parámetro incorrecto. Este ejemplo muestra que la validación no es un trámite burocrático: es una herramienta de depuración esencial.

10. Ejemplo escalado: celda de manufactura $M/E_2/1$ con análisis de sensibilidad

Sistema

Una celda de manufactura recibe piezas que llegan según un proceso de Poisson con $\lambda = 8$ piezas/h. El tiempo de procesamiento consiste en dos etapas secuenciales (carga y mecanizado) de duración exponencial con tasa $\mu_{\text{fase}} = 10$ operaciones/h cada una, por lo que $S \sim E_2(10)$, con $\mathbb{E}[S] = 0{,}2$ h y $\text{CV}_S = 1/\sqrt{2}$.
El modelo conceptual es una cola $M/E_2/1$. Se aplica el proceso completo de V&V, incluyendo verificación, validación contra la fórmula P-K, y análisis de sensibilidad.

Parámetros del sistema:

$$\mathbb{E}[S] = \frac{2}{10} = 0{,}2 \text{ h}, \quad \mathrm{Var}[S] = \frac{2}{10^2} = 0{,}02 \text{ h}^2, \quad \rho = 8 \times 0{,}2 = 0{,}80$$

$$\mathbb{E}[S^2] = \mathrm{Var}[S] + (\mathbb{E}[S])^2 = 0{,}02 + 0{,}04 = 0{,}06 \text{ h}^2$$

10.1. Verificación estructural

Prueba de tráfico extremo: con $\lambda = 9{,}9$ piezas/h ($\rho = 0{,}99$), la cola crece sin límite. En una corrida de $T = 500$ h, L_q crece monótonamente y supera 200 piezas. ✓

Prueba de simplificación: un servicio $E_2(\mu_f)$ es equivalente a dos fases exponenciales en serie. Se verifica que la distribución de $S_1 + S_2$ tiene media 0,2 h, varianza 0,02 h^2 y asimetría $2/\sqrt{2} \approx 1{,}41$ (valor teórico de la Erlang-2). ✓

Conservación: en $T = 2000$ h con $\lambda = 8$, se esperan $\approx 16\,000$ llegadas. La simulación registra 16 023 llegadas y 16 023 salidas (sistema vacío al final). ✓

10.2. Validación operacional con la fórmula P-K

La fórmula de Pollaczek–Khinchine da:

$$W_q^{\text{P-K}} = \frac{\lambda \mathbb{E}[S^2]}{2(1-\rho)} = \frac{8 \times 0{,}06}{2 \times 0{,}2} = \frac{0{,}48}{0{,}4} = 1{,}20 \text{ h}$$

Para comparar: si el servicio fuera exponencial ($M/M/1$, misma $\mathbb{E}[S]$):

$$W_q^{M/M/1} = \frac{\rho}{\mu(1-\rho)} = \frac{0{,}8}{5 \times 0{,}2} = 1{,}60 \text{ h}$$

El servicio Erlang-2 reduce el tiempo en cola en un 25 % gracias a la menor variabilidad (CV = 0,71 vs. 1).

Resultados de simulación ($n_R = 50$ réplicas, $T = 2000$ h):

Métrica	**P-K / analítico**	**Simulado**	**IC 95 %**	**¿Contiene ref.?**
W_q (h)	1,200	1,214	[1,176; 1,252]	Sí ✓
L_q (pzas)	9,600	9,712	[9,408; 10,016]	Sí ✓
ρ	0,800	0,799	[0,795; 0,803]	Sí ✓

Todos los IC contienen el valor de referencia. El modelo está validado.

10.3. Análisis de sensibilidad

Se varía λ en ± 10 % y ± 20 % alrededor del valor base ($\lambda_0 = 8$):

λ	ρ	$W_q^{\text{P-K}}$ (h)	$\bar{W}_q^{\text{sim}}$ (h)	**Dirección**
6,4 (−20 %)	0,64	0,53	0,55	✓
7,2 (−10 %)	0,72	0,77	0,79	✓
8,0 (base)	0,80	1,20	1,21	—
8,8 (+10 %)	0,88	2,20	2,24	✓
9,6 (+20 %)	0,96	7,20	7,45	✓

Observaciones:

- La **dirección** es correcta en todos los casos: W_q aumenta con λ. ✓
- La **no-linealidad** es correcta: un aumento del 20 % en λ (de 8 a 9,6) produce un aumento del 500 % en W_q (de 1,2 a 7,2 h). Esto es consistente con la teoría de colas: cerca de la saturación ($\rho \to 1$), el sistema es extremadamente sensible. ✓
- Todos los valores simulados caen dentro del 4 % del valor P-K. ✓

Conclusión de V&V

El proceso de verificación confirmó la correcta implementación del modelo (conservación de entidades, trazas correctas, pruebas extremas consistentes). La validación operacional demostró que el simulador reproduce las métricas analíticas dentro del margen estadístico esperado (error relativo $< 1{,}2\,\%$). El análisis de sensibilidad confirmó que el modelo responde correcta y coherentemente ante variaciones de λ.
Esto otorga credibilidad para usar el modelo en análisis que exceden las capacidades analíticas: percentiles de tiempo de espera, políticas de prioridad, configuraciones con múltiples colas.

Referencias

[1] Law, A. M. (2015). *Simulation Modeling and Analysis*, 5th ed. McGraw-Hill. Cap. 5 y 10.

[2] Sargent, R. G. (2013). Verification and validation of simulation models. *Journal of Simulation*, 7(1), 12–24.

[3] Banks, J., Carson, J. S., Nelson, B. L. & Nicol, D. M. (2014). *Discrete-Event System Simulation*, 5th ed. Pearson. Cap. 10.

[4] Balci, O. (1998). Verification, validation, and testing. En *Handbook of Simulation*. Wiley. Cap. 10.

[5] Kelton, W. D., Sadowski, R. P. & Zupick, N. B. (2015). *Simulation with Arena*, 6th ed. McGraw-Hill. Cap. 10.

[6] Schlesinger, S. et al. (1979). Terminology for model credibility. *Simulation*, 32(3), 103–104.